UKAPI.io

Senior Engineer Architecture & Implementation Report

Laravel architecture, provider integrations, metering, resilience, security and launch plan

Prepared 23 September 2026

Primary stack: Laravel 13 / PHP 8.3+

Simple UK data. One API.

CONFIDENTIAL WORKING DOCUMENT

Version: 1.0 **Date:** 23 September 2026 **Audience:** Senior/lead engineer implementing UKAPI.io
Primary stack: Laravel 13, PHP 8.3+, MySQL, Redis

1. Purpose

This document turns the UKAPI.io MVP concept into an implementation blueprint. It assumes a small engineering team or a single experienced developer and intentionally favours a modular Laravel monolith over distributed services.

The architectural objective is not merely to proxy upstream APIs. UKAPI.io needs to own a stable external contract while upstream providers remain replaceable implementation details. The critical boundaries are therefore domain contracts, provider adapters, caching, metering, authentication and failure isolation.

2. Engineering principles

1. **Public contracts are ours; provider contracts are not.** Controllers return UKAPI.io resources/DTOs, never provider payloads.
2. **Monolith first.** One Laravel deployment, one relational database and one Redis cluster are enough for MVP.
3. **Provider isolation.** Every external source lives behind an interface plus adapter.
4. **Cache by semantics.** Cache keys represent normalized requests, not raw URLs.
5. **Fail boundedly.** Every upstream call has strict timeouts, limited retries and a known fallback policy.
6. **Meter at the gateway.** Usage accounting happens consistently after authentication and route resolution.
7. **Keep billing out of request hot paths.** Request authorization uses locally stored entitlements, not Stripe network calls.
8. **Observable by default.** Every request has a request ID and endpoint/provider metrics.
9. **Schema stability beats raw completeness.** Expose useful fields deliberately instead of mirroring every source property.
10. **Geographic scope is data.** England-only, GB-only and UK-wide differences must be represented explicitly.

3. Recommended runtime and dependencies

3.1 Core

- Laravel 13 (laravel/framework ^13.0)
- PHP 8.3 or newer
- MySQL 8.x-compatible managed database
- Redis 7.x-compatible managed service
- Nginx + PHP-FPM, FrankenPHP, Laravel Cloud or another supported Laravel deployment target
- Composer 2
- Node/Vite for the dashboard assets

Laravel 13 is the current major release as of September 2026 and requires PHP 8.3+.

3.2 Laravel packages/features

Use first-party capabilities where practical:

- Queue workers + Horizon
- Cache and Redis rate limiting
- Laravel HTTP client
- Laravel Cashier for Stripe billing
- Notifications/mail
- Scheduler
- Policies/gates
- encrypted casts / framework encryption where appropriate

Sanctum is suitable for authenticated dashboard/API interactions belonging to logged-in users, but customer API keys should use a dedicated ApiKey model because UKAPI.io needs explicit prefixes, scopes, quotas, revocation metadata and high-speed hash lookup.

3.3 Testing

Use Pest 4 or PHPUnit 12. The test suite should include unit tests, feature tests, contract tests for adapters and a smaller set of live-provider smoke tests that are not required on every CI run.

4. Repository structure

Avoid a generic “Services” dumping ground. Group provider and API-domain code explicitly.

UKAPI.io

```
app/
  Console/
  Domain/
    ApiKeys/
    Billing/
    Companies/
    Crime/
    FoodHygiene/
    Flood/
    Geography/
    Planning/
    Usage/
  Http/
    Controllers/
      Api/V1/
      Dashboard/
      Admin/
    Middleware/
    Requests/
    Resources/
  Integrations/
    CompaniesHouse/
    EnvironmentAgency/
    FoodStandardsAgency/
    GovUk/
    PlanningData/
    PoliceUk/
    PostcodesIo/
  Jobs/
  Models/
  Providers/
  Support/
    Api/
    Cache/
    Http/
    Observability/
config/
  ukapi.php
routes/
  api.php
  web.php
  console.php
openapi/
  openapi.yaml
```

Domain folders own UKAPI.io concepts; Integrations owns source-specific code.

5. Request lifecycle

A typical API request should flow as follows:

```
Client
 -> CDN/WAF (optional)
 -> Laravel route
 -> AssignRequestId middleware
 -> ApiKeyAuthentication middleware
 -> ApiKeyStatus middleware
 -> BurstRateLimit middleware
 -> MonthlyQuota middleware
 -> request validation
 -> application/domain service
 -> normalized cache lookup
 -> provider adapter if cache miss
 -> provider response validation
 -> normalizer/DTO
 -> cache write
 -> API Resource
 -> usage event / metrics
 -> response
```

The request path must not synchronously contact Stripe.

6. API key design

6.1 Key format

Use recognizable random credentials:

```
uk_live_<32+ random bytes encoded safely>
uk_test_<32+ random bytes encoded safely>
```

Do not use sequential IDs or reversible encrypted values as the credential.

6.2 Storage

Store:

- short non-secret key prefix/identifier for lookup/display;
- cryptographic hash of the secret token;
- user/team owner;
- environment (live, test);
- name;
- scopes if implemented;
- status;
- created/last-used/revoked timestamps;
- optional last-used IP truncated/retained according to privacy policy.

A performant pattern is to embed a random public key ID in the credential and hash only the secret portion. That allows indexed lookup followed by constant-time verification.

Example conceptual token:

```
uk_live_k_8F3K2A.s_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Database stores k_8F3K2A plus a hash of the secret. Avoid scanning hashes.

6.3 Table

```
api_keys
- id bigint/uuid
- user_id
- public_id unique
- secret_hash
- name
- environment
- status
- scopes json nullable
- last_used_at nullable
- revoked_at nullable
- created_at
- updated_at
```

6.4 Authentication middleware

Pseudo-code:

```
$token = BearerToken::fromRequest($request);
$parts = ApiKeyToken::parse($token);
$key = ApiKey::query()->where('public_id', $parts->publicId)->first();

if (! $key || ! $key->isActive()) {
    throw ApiException::unauthorized('invalid_api_key');
}

if (! Hash::check($parts->secret, $key->secret_hash)) {
    throw ApiException::unauthorized('invalid_api_key');
}

$request->attributes->set('api_key', $key);
```

Use an appropriate password/hash primitive supported by Laravel. API secrets are high entropy, so even a fast keyed HMAC comparison design can be appropriate if implemented carefully; using framework hashing is simpler and safer for MVP, with the public ID preventing expensive bulk scans.

7. Plans, entitlements and quotas

Separate the commercial plan from effective runtime entitlements.

Suggested tables:

```
plans
- id
- code
- name
- monthly_price_pence
- monthly_request_quota
- burst_rps
- active

subscriptions
- id
- user_id
- plan_id
- stripe_subscription_id nullable
- status
- period_starts_at
- period_ends_at

entitlements
- user_id
- monthly_request_quota
- burst_rps
- endpoint_overrides json nullable
- effective_until nullable
```

Stripe webhooks update local subscription state and entitlements. API middleware reads local data cached in Redis.

Do not make a network request to Stripe to decide whether an API request is allowed.

8. Usage metering architecture

8.1 Requirements

Metering must be fast, resilient and reconcilable. Exact-once distributed accounting is unnecessary at MVP scale, but silent large undercounting is unacceptable.

8.2 Hot-path counter

Increment a Redis counter after a request has passed authentication and is accepted for processing:

```
usage:{YYYYMM}:{api_key_id} -> integer
usage-endpoint:{YYYYMM}:{api_key_id}:{endpoint_code} -> integer
```

Set expiry beyond the billing/reconciliation window.

8.3 Durable rollup

A scheduled job periodically persists Redis deltas or aggregates request usage into MySQL.

Suggested table:

```
api_usage_daily
- id
- usage_date
- api_key_id
- endpoint_code
- request_count
- success_count
- client_error_count
- server_error_count
- cache_hit_count
- total_duration_ms
UNIQUE (usage_date, api_key_id, endpoint_code)
```

Do not insert one SQL row per API request unless request-log functionality explicitly requires it; that becomes needless write amplification.

8.4 Request logs

If exposing recent request history, maintain a short-lived sampled/compact request log separately:

```
api_request_logs
- id/ulid
- request_id
- api_key_id
- endpoint_code
- status_code
- duration_ms
- cached
- provider_code nullable
- created_at
```

Avoid storing full query values for sensitive or potentially identifying endpoints. Consider a 7-30 day retention window.

9. Rate limiting

Implement two independent limits:

11. burst rate in Redis;
12. monthly entitlement counter.

Laravel's Redis-backed limiter can implement burst windows. Return standard rate-limit headers where useful:

```
RateLimit-Limit
RateLimit-Remaining
RateLimit-Reset
Retry-After
```

A 429 response should use the normal UKAPI.io error envelope.

Plan checks should be keyed by API key, not user, so multiple applications remain independently observable. The monthly quota can either be per account or key depending on product semantics;

per-account is simpler commercially, while per-key is simpler operationally. Recommendation: **account-level quota, key-level rate limits**.

10. Public response model

10.1 Envelope

Implement shared API response helpers/resources, but avoid wrapping internal Laravel paginator structures directly.

```
{
 "data": {},
 "meta": {
  "request_id": "req_...",
  "cached": false,
  "source": "food_standards_agency",
  "retrieved_at": "2026-09-23T18:00:00Z",
  "source_updated_at": null,
  "coverage": "UK"
 }
}
```

10.2 Error codes

Maintain an enumerated machine-readable code list, for example:

```
invalid_api_key
api_key_revoked
quota_exceeded
rate_limit_exceeded
invalid_postcode
postcode_not_found
company_not_found
unsupported_geography
invalid_parameter
upstream_unavailable
upstream_timeout
upstream_invalid_response
internal_error
```

Error messages can evolve; codes should remain stable within `/v1`.

10.3 Request IDs

Generate a ULID/UUID-style request ID at ingress or honour a valid edge-generated ID. Include it in:

- response header (`X-Request-Id`);
- response `meta` or `error` body;
- structured logs;
- provider-call logs/metrics.

11. Provider adapter contract

Define narrow interfaces per domain rather than one universal “DataProvider”.

Example:

```
interface CompanyProvider
{
    public function find(string $companyNumber): CompanyData;

    /** @return CompanySearchResult[] */
    public function search(string $query, int $page, int $perPage): array;

    /** @return OfficerData[] */
    public function officers(string $companyNumber): array;
}
```

CompaniesHouseCompanyProvider implements it. Controllers depend on a domain service, not the adapter directly.

DTOs should be immutable value objects where practical. Laravel API Resources transform DTOs/domain objects into external JSON.

12. HTTP client policy

Create a shared integration HTTP factory/wrapper that standardizes:

- user agent identifying UKAPI.io;
- connect timeout;
- total timeout;
- JSON headers;
- request ID forwarding where appropriate;
- retry policy;
- telemetry;
- redaction.

Example policy:

```
Http::baseUrl($baseUrl)
    ->acceptJson()
    ->connectTimeout(2)
    ->timeout(5)
    ->retry(
        times: 2,
        sleepMilliseconds: 150,
        when: fn ($exception, $request) => RetryPolicy::isSafe($exception)
    );
```

Do not retry most 4xx responses. Retry transient connection failures and selected 429/5xx responses only when the endpoint and upstream semantics make it safe.

Keep maximum total upstream wall time bounded.

13. Provider-specific notes

13.1 Postcodes.io / geography

The initial geography adapter can use Postcodes.io for lookup/reverse lookup. It requires no authentication and exposes ONS-linked administrative geography.

Cache normalized postcode records for a long TTL. Canonicalize user input before key generation:

```
"bl2 6xx" -> "BL26XX" cache key
```

Store the canonical display form in the response.

Do not expose every raw ONS field initially. Define a versioned PostcodeData DTO containing fields UKAPI.io commits to supporting.

13.2 Companies House

Companies House public data requires authentication credentials. Keep its key in encrypted secret configuration, not the database unless rotation needs demand it.

Important implementation points:

- normalize company numbers (including letter prefixes);
- preserve authoritative company number as a string;
- cache company profiles briefly;
- apply provider-specific throttling below upstream hard limits;
- cache 404 results for a short negative TTL to prevent repeated misses;
- validate date formats and optional fields;
- do not couple public pagination to Companies House's exact pagination model.

13.3 GOV.UK Bank Holidays

This can be cached heavily. Fetch on schedule into a local snapshot and serve requests without upstream dependency in the hot path.

A scheduled command can refresh daily and alert on parse/schema failure.

13.4 Food Standards Agency

FHRS/FHIS rating models differ geographically. Model the public response around an explicit scheme:

```
{
 "scheme": "FHRS",
 "rating": {
  "value": "5",
  "label": "Very good"
 }
}
```

Do not coerce every scheme to a fake 0-5 integer.

Provider results include geocoded establishments and can support postcode/nearby searches. Cache search responses moderately and establishment details longer.

13.5 Police.uk

Street crime is coordinate-driven and locations are approximate/anonymised. A postcode request should internally resolve postcode coordinates first.

Implementation flow:

```
postcode -> GeographyService -> lat/lng -> PoliceUkProvider -> CrimeNormalizer
```

The public meta should state requested month and coverage caveat. For Scotland, return explicit coverage metadata rather than silently implying completeness.

Crime history is effectively immutable after publication except for revisions, so older monthly data can use long TTLs.

13.6 Environment Agency flood data

This source is England-specific. Near-real-time data needs short TTLs and timestamps.

Use stale cache conservatively: a stale river-level reading may be acceptable if visibly timestamped; a stale active warning may be safety-relevant. Do not present UKAPI.io as an emergency alert service and include official-source attribution.

13.7 Planning Data

The Planning Data API exposes many England datasets and currently carries beta caveats in its documentation. Do not build public endpoint names around raw dataset internals.

Create an internal set of constraint checkers:

```
ConservationAreaConstraint
ListedBuildingConstraint
GreenBeltConstraint
Article4Constraint
FloodRiskConstraint
```

Return a stable structure such as:

```
{
 "type": "conservation_area",
 "matched": true,
 "entities": [
  { "reference": "...", "name": "..." }
 ]
}
```

The service can expand the source dataset mapping without changing the public concept.

14. Cache design

14.1 Cache layers

MVP requires only Redis application cache. CDN caching can later be added for endpoints whose auth/plan model permits it.

14.2 Key format

```
ukapi:v1:{endpoint-code}:{normalized-parameter-hash}
```

Examples:

```
ukapi:v1:postcode:BL26XX
ukapi:v1:company:00445790
ukapi:v1:crime-summary:BL26XX:2026-08
```

14.3 Stampede protection

Use a short Redis lock on expensive misses:

```
return Cache::lock("lock:$key", 10)->block(2, function () use ($key) {
    return Cache::remember($key, $ttl, fn () => $this->fetchAndNormalize());
});
```

Where blocking would harm latency, return stale content and queue refresh instead.

14.4 Stale-while-revalidate

Maintain fresh and stale horizons conceptually:

```
fresh_until = 15 minutes
stale_until = 2 hours
```

If fresh: serve. If stale-but-allowed: serve and dispatch refresh job. If expired: synchronously fetch or fail based on endpoint policy.

Store cache metadata alongside payload:

```
{
  "payload": {},
  "retrieved_at": "...",
  "source_updated_at": "...",
  "fresh_until": "...",
  "stale_until": "..."
}
```

15. Circuit breaking

Laravel does not require a heavyweight circuit-breaker package for MVP. Implement a simple Redis-backed provider health state:

```
provider-health:{provider}:failures
provider-health:{provider}:open-until
```

If a provider exceeds a threshold of recent transient failures, temporarily stop synchronous upstream calls and either return safe stale cache or a fast 503.

Half-open behaviour can permit a small number of probe requests after cooldown.

Circuit states should be visible in the admin dashboard.

16. Database schema

Core tables:

```
users
api_keys
plans
subscriptions
entitlements
api_usage_daily
api_request_logs (optional/short retention)
provider_health_events
provider_snapshots (optional for local snapshots)
feature_flags
admin_audit_logs
```

16.1 Users

Laravel normal user fields plus email verification and optional two-factor fields depending on chosen auth starter.

16.2 Provider configuration

Secrets should normally live in environment/secret management rather than a generic database table. Database configuration may store non-secret flags such as provider enabled/disabled or override TTLs.

16.3 Feature flags

Simple schema:

```
feature_flags
- key unique
- enabled boolean
- value json nullable
- updated_by nullable
- timestamps
```

Examples:

```
endpoint.crime.enabled
endpoint.planning.enabled
provider.companies-house.stale-fallback
```

17. Routing and endpoint identifiers

Routes should attach a stable internal endpoint code used for metering and docs.

Laravel 13 supports route metadata, but an explicit custom route default/middleware mapping is also acceptable. Conceptually:

```
Route::get('/v1/postcodes/{postcode}', PostcodeController::class)
    ->name('api.v1.postcodes.show')
    ->metadata([
        'ukapi' => [
            'endpoint' => 'postcodes.show',
            'provider' => 'postcodes_io',
        ],
    ]);
```

Do not derive billing identity from controller class names because refactors would corrupt usage reporting.

18. Controllers, actions and resources

Keep controllers thin:

```
final class ShowCompanyController
{
    public function __invoke(ShowCompanyRequest $request, CompanyLookup $lookup): CompanyResource
    {
        $company = $lookup->handle($request->companyNumber());

        return CompanyResource::make($company);
    }
}
```

`CompanyLookup` owns cache/provider/fallback orchestration. `CompanyResource` owns public serialization.

This separation makes contract testing straightforward.

19. Validation and normalisation

Create value objects for inputs that recur:

- `UkPostcode`
- `CompanyNumber`
- `YearMonth`
- `Latitude`
- `Longitude`
- `RadiusKm`

Example:

UKAPI.io

```
final readonly class UkPostcode
{
    public static function from(string $value): self
    {
        $normalized = strtoupper(preg_replace('/\\s+/', '', trim($value)));

        if (! PostcodePattern::isValid($normalized)) {
            throw InvalidPostcode::make($value);
        }

        return new self($normalized);
    }
}
```

Syntax validation and “exists in source data” are distinct states and should generate different behaviour where useful.

20. OpenAPI strategy

Treat OpenAPI as a deliverable and contract, not an incidental generated dump.

Recommended workflow:

13. maintain openapi/openapi.yaml in source control;
14. validate it in CI;
15. use it to power documentation/reference pages;
16. generate typed SDKs later;
17. add contract tests verifying key Laravel responses conform to schemas.

A fully annotation-driven spec can become noisy. For this product, a hand-curated OpenAPI document with reusable components is likely clearer.

Version the public API in path (/v1) and the OpenAPI document independently by build/revision.

21. Billing implementation

Use Stripe Checkout/Customer Portal or equivalent Cashier-supported flows to minimise custom payment UI.

Stripe webhooks to handle:

- checkout completion;
- subscription created/updated/deleted;
- invoice paid;
- invoice payment failed.

Webhook handlers must be idempotent and verify signatures.

Flow:

```
Stripe webhook
 -> verify signature
 -> idempotency/event store check
 -> update local subscription
 -> recalculate entitlements
 -> invalidate entitlement cache
```

Never immediately delete API access on the first transient payment failure unless that is an explicit product policy. Model grace periods in subscription state.

22. Queue jobs and scheduler

Likely jobs:

```
RefreshProviderCache
RefreshBankHolidaySnapshot
RollUpApiUsage
PruneRequestLogs
ReconcileSubscriptionEntitlements
ProbeProviderHealth
SendUsageThresholdNotification
WarmPopularCaches
```

Scheduler examples:

- roll up usage every 5 minutes;
- provider health probe every minute or five minutes;
- bank holiday refresh daily;
- prune logs daily;
- subscription reconciliation daily;
- backup jobs according to hosting strategy.

Queue provider calls only where asynchronous semantics make sense. A user waiting for a synchronous lookup should not be forced into job polling unless the operation is genuinely long-running.

23. Observability

23.1 Structured logs

Emit JSON logs in production with fields such as:

```
request_id
api_key_id (internal ID only)
user_id
endpoint_code
provider_code
status_code
duration_ms
cache_status
upstream_status
upstream_duration_ms
error_code
```

Never log API secret values, provider secret values, authorization headers or Stripe secrets.

23.2 Metrics

At minimum measure:

- request count by endpoint/status;
- latency histograms;
- provider latency/error count;
- cache hit/miss/stale counts;
- rate-limit count;
- quota rejection count;
- queue depth/failures;
- DB/Redis health;
- provider circuit state.

23.3 Error tracking

Send unhandled exceptions to an error tracker with request ID and safe contextual tags. Scrub headers and query values where necessary.

23.4 Health endpoints

Expose two concepts:

```
/up      -> process is alive, no expensive dependencies
/health  -> internal/secured deeper dependency status
```

Public load balancers should not depend on every external provider being healthy, otherwise one government API outage could remove all UKAPI.io instances from rotation.

24. Security design

24.1 Threats

Primary MVP threats include:

- leaked customer API keys;
- credential stuffing against dashboard accounts;
- request floods causing upstream quota exhaustion;
- malicious parameter values attempting SSRF/path injection;
- source payload poisoning or malformed data;
- log leakage;
- admin account compromise;
- abuse of aggregation to enumerate data at high volume.

24.2 Controls

- fixed provider base URLs; never let customers provide an upstream URL;
- strict scalar validation and result caps;
- API key hashing;

- admin 2FA;
- secure session cookies;
- CSRF on dashboard;
- CSP and normal Laravel security headers;
- dependency scanning;
- WAF/CDN optional but useful;
- per-IP pre-auth limits;
- per-key post-auth limits;
- provider-specific internal limits;
- encrypted secrets using deployment secret store;
- least-privilege DB and infrastructure credentials;
- audit logs for privileged admin changes.

24.3 SSRF

Because this product calls many external APIs, integration code is a sensitive SSRF surface. Provider URLs must be configured constants; do not construct hostnames from user input. Redirect handling should be disabled or constrained when possible.

25. Privacy and data retention

UKAPI.io primarily handles public-source data, but customer usage data is still personal/business data.

Recommended retention defaults:

- aggregate usage: long-term for billing/analytics as justified;
- detailed request logs: 7-30 days;
- security/audit events: longer, e.g. 6-12 months, depending on policy;
- revoked API key hash: retain as needed for audit but secret cannot be recovered;
- full client IP: minimise or truncate unless security requirements justify temporary retention.

Provide account deletion workflows that preserve records you are legally required to retain while removing unnecessary personal data.

26. Source licensing register

Create a repository document or database-backed internal register before shipping each provider.

Suggested fields:

```
provider_code
owner
terms_url
licence
required_attribution
commercial_reuse_allowed
cache_allowed
redistribution_notes
geographic_scope
reviewed_at
reviewed_by
```

Make provider activation conditional on a completed source review in engineering workflow.

27. Geographic coverage model

Avoid ambiguous booleans like `uk_wide`.

Represent coverage explicitly:

```
UK
Great Britain
England and Wales
England
Wales
Scotland
Northern Ireland
Provider-specific partial coverage
```

Include `coverage` in API metadata where it materially affects interpretation.

A future `/v1/meta/endpoints` endpoint can expose endpoint availability and coverage programmatically.

28. Admin/operations implementation

Admin screens should provide:

- user/account search;
- plan and entitlement view;
- API key revocation;
- usage graphs;
- top endpoints;
- provider status;
- circuit-breaker state;
- recent server errors;
- cache metrics;
- endpoint feature flags;
- manual quota grants;
- subscription state;
- audit log.

Every privileged mutation should create an admin_audit_logs entry containing actor, action, target, before/after summary and request ID.

29. Testing strategy

29.1 Unit tests

Test:

- value objects and input normalisation;
- provider payload normalizers;
- cache policy decisions;
- quota calculations;
- error mappings;
- rating-scheme mappings;
- geographic coverage rules.

29.2 Provider contract tests

Use captured representative fixtures from each upstream provider. Test adapters against those fixtures without network calls.

Fixtures should include:

- normal response;
- missing optional fields;
- empty list;
- not found;
- upstream 429;
- upstream 500;
- invalid JSON/schema drift where possible.

29.3 Feature tests

Test complete Laravel requests:

- valid key;
- invalid/revoked key;
- rate limit;
- monthly quota;
- cache hit;
- cache miss;
- provider timeout;
- stale fallback;
- validation errors;
- pagination;
- request ID propagation.

29.4 Billing tests

Use Stripe test mode. Test signed webhooks and duplicate webhook delivery.

29.5 Live smoke tests

Run scheduled production/staging probes against a few known stable records, for example one postcode and a known company. Alert on schema changes before customers report them.

Do not make the ordinary pull-request CI pipeline dependent on government API availability.

30. CI/CD

Suggested pipeline:

```
composer validate
composer install
lint/style checks
static analysis
unit + feature tests
OpenAPI validation
frontend build/tests
security/dependency audit
build deploy artifact/container
migrate with safe strategy
deploy
post-deploy smoke test
```

Consider Laravel Pint for style and PHPStan/Larastan for static analysis.

Deployments should be reproducible and environment configuration should live outside the repository.

31. Database migration policy

Design migrations for rolling deployment where practical:

18. additive schema change;
19. deploy code supporting old/new state;
20. backfill asynchronously if required;
21. switch reads/writes;
22. remove obsolete schema in a later deploy.

At MVP traffic, this may feel excessive, but it prevents avoidable downtime as usage grows.

32. Performance targets

Initial internal targets, not contractual SLAs:

- cached endpoint p95 application latency: <150 ms excluding internet edge overhead;
- uncached typical upstream endpoint p95: <1.5 s where provider permits;
- API authentication/rate-limit overhead: <20 ms on healthy Redis/DB cache path;

- cache hit ratio: >80% overall after traffic warms, with much higher ratios for immutable/reference endpoints;
- platform-generated 5xx rate: <0.5% under normal upstream conditions.

Provider latency and outages should be measured separately from UKAPI.io platform faults.

33. Scalability path

Do not pre-emptively split services. Scaling sequence should be:

23. multiple stateless Laravel app instances;
24. managed Redis and DB sizing;
25. dedicated queue worker pools;
26. read replicas if analytics/dashboard reads justify them;
27. provider-specific concurrency controls;
28. CDN/edge caching for safe public/cacheable content;
29. move heavy snapshots/search datasets to dedicated stores only when evidence supports it.

Possible future service extraction candidates are usage analytics and bulk dataset ingestion, but only after they demonstrably constrain the monolith.

34. Search strategy

Do not introduce Elasticsearch/OpenSearch for MVP unless an endpoint truly needs it. Companies House/FSA searches can initially rely on upstream search. Local SIC/council metadata can use MySQL indexes/full-text or small in-memory/cached datasets.

If UKAPI.io later ingests national datasets and provides fuzzy cross-source search, then a dedicated search engine can be justified.

35. Deployment topology

A sensible production topology:

```
Cloudflare / CDN / WAF (optional)
        |
Load balancer / platform router
        |
Laravel web instances
        |--- MySQL
        |--- Redis
        |--- Queue workers / Horizon
        |--- S3-compatible storage (if required)
        |
External providers
```

Keep staging separate from production credentials and quotas.

36. Backups and disaster recovery

At minimum:

- automated daily database backups with point-in-time recovery where available;
- Redis treated primarily as disposable cache/counter infrastructure, but understand metering implications of loss;
- infrastructure/configuration documented or codified;
- restore procedure tested before launch;
- Stripe remains payment system of record, but local subscription reconciliation must be able to rebuild state after recovery.

Usage counters in Redis should be flushed to durable storage frequently enough that a Redis loss cannot materially affect a billing cycle.

37. Incident handling

Define incident classes:

- platform outage;
- provider outage;
- degraded endpoint family;
- incorrect/stale data;
- billing issue;
- suspected credential/security incident.

For provider outages, the preferred response is to disable/degrade only the affected endpoint family rather than the whole API.

Maintain a simple runbook containing provider disable commands/flags, queue restart procedure, cache invalidation, credential rotation and customer-status communication steps.

38. Source freshness policy

Each response family should have a documented freshness contract expressed as **how UKAPI.io behaves**, not a guarantee about upstream publication.

Example:

```
postcode: cache up to 30 days
company profile: refresh after 15 minutes
bank holiday snapshot: refresh daily
crime historical month: refresh daily initially, then extend TTL after ageing
flood warning: refresh every 5 minutes
planning constraints: refresh daily
```

Return timestamps so the caller can make its own decision.

39. Negative caching

Cache not-found responses briefly to prevent abusive/repetitive misses:

- invalid syntax: do not call provider at all;
- syntactically valid but nonexistent postcode: 5-60 minutes;
- company not found: 5-15 minutes;

- provider temporary error: never cache as a permanent not-found.

Keep negative-cache keys distinct from successful payloads.

40. Idempotency

Most public MVP endpoints are GET and naturally idempotent. For dashboard mutations such as API-key creation and future batch jobs, optional idempotency keys can be added later.

Stripe webhooks must always be idempotent based on Stripe event ID.

41. Documentation implementation

Documentation should combine narrative guides and generated/reference material.

Suggested structure:

```
/docs
 /quickstart
 /authentication
 /errors
 /rate-limits
 /pagination
 /sources-and-coverage
 /api/postcodes
 /api/companies
 ...
```

Code samples should include C# prominently because UK utility APIs are commonly used by .NET business applications, alongside PHP, JavaScript and cURL.

Example C#:

```
using var client = new HttpClient
{
   BaseAddress = new Uri("https://api.ukapi.io/v1/")
};

client.DefaultRequestHeaders.Authorization =
   new AuthenticationHeaderValue("Bearer", apiKey);

var response = await client.GetAsync("postcodes/BL26XX");
response.EnsureSuccessStatusCode();
```

42. SDK plan

Do not block launch on SDKs. OpenAPI plus standard HTTP examples are sufficient.

First SDK candidates after launch:

30. PHP/Laravel
31. C#/.NET
32. TypeScript
33. Python

SDKs should remain thin wrappers around the public contract and be generated/part-generated from OpenAPI where practical.

43. Suggested Laravel interfaces/classes

```
App\Support\Api\ApiRequestContext
App\Support\Api\ApiResponse
App\Support\Api\ApiException
App\Domain\ApiKeys\ApiKeyAuthenticator
App\Domain\Usage\UsageMeter
App\Domain\Usage\QuotaService
App\Domain\Geography\GeographyService
App\Domain\Companies\CompanyLookup
App\Domain\Crime\CrimeLookup
App\Domain\Planning\PlanningConstraintService
App\Integrations\Contracts\... (domain-specific provider interfaces)
```

Use dependency injection bindings in service providers so a provider can be swapped without controller changes.

44. Example Company implementation

```
final class CompanyLookup
{
    public function __construct(
        private CompanyProvider $provider,
        private Repository $cache,
    ) {}

    public function find(CompanyNumber $number): CompanyData
    {
        $key = 'ukapi:v1:company:'.$number->value();

        return $this->cache->remember(
            $key,
            now()->addMinutes(15),
            fn () => $this->provider->find($number->value()),
        );
    }
}
```

Provider:

```
final class CompaniesHouseCompanyProvider implements CompanyProvider
{
    public function __construct(private PendingRequest $http) {}

    public function find(string $companyNumber): CompanyData
    {
        $response = $this->http
            ->get("/company/{$companyNumber}")
            ->throw();

        return CompaniesHouseMapper::company($response->json());
    }
}
```

In production, wrap source errors rather than calling `throw()` blindly into controllers; the snippet only illustrates boundaries.

45. Provider error mapping

Create provider-specific exception translation:

```
upstream 404 -> domain NotFound -> public 404
upstream 400 caused by our bad mapping -> internal/provider integration error
upstream 429 -> transient provider throttled -> stale fallback or 503
upstream 500/502/503 -> transient provider failure -> retry/fallback
connect timeout -> upstream_timeout
invalid JSON/schema -> upstream_invalid_response
```

Do not tell customers “Companies House returned 502” as the primary API contract. Put source details in internal telemetry and expose a generic provider code only where useful.

46. Concurrency controls

A sudden popular client could cause UKAPI.io to exceed an upstream provider rate limit even if each UKAPI.io plan limit appears reasonable.

Use a provider-wide limiter in addition to customer limits:

```
provider-limit:companies-house
provider-limit:police-uk
```

Cache coalescing/locks reduce duplicate identical upstream requests.

47. Data snapshots

Some sources are better served as periodically refreshed snapshots instead of live pass-through calls, especially stable reference datasets.

Candidate snapshot data:

- bank holidays;
- SIC codes;
- council metadata;

- possibly postcode/reference data later if self-hosting is justified.

Store snapshot version/source timestamp and make refresh atomic: load new snapshot, validate it, then switch active version.

48. Versioning policy

/v1 commits to field semantics, error codes and endpoint behaviour.

Within v1:

- adding nullable fields is normally non-breaking;
- adding new enum values may break poorly written clients, so document forward compatibility;
- renaming/removing fields is breaking;
- changing meaning/units is breaking;
- changing source provider is not breaking if public semantics stay equivalent.

When v2 is eventually needed, operate v1 and v2 in parallel for a communicated migration period.

49. Pagination policy

Use one UKAPI.io pagination model regardless of provider. Prefer cursor pagination for large mutable collections, but simple page/per-page is acceptable for MVP search endpoints if upstream APIs naturally support it.

Cap per_page (for example at 100) to prevent amplification abuse.

50. Date/time conventions

- ISO 8601 timestamps in UTC (Z) for timestamps;
- ISO YYYY-MM-DD for date-only values;
- YYYY-MM for crime periods;
- explicitly document UK-local calendar concepts like bank-holiday divisions;
- do not return ambiguous human-formatted dates in machine fields.

51. Money/number conventions

If future endpoints include money, expose integer minor units where precision matters and ISO currency codes. For the VAT calculator, accept decimal strings/numbers carefully and use decimal arithmetic rather than binary floating point.

52. Accessibility and frontend quality

Although primarily an API product, the docs/dashboard are part of the developer experience. Meet WCAG 2.2 AA as a practical target: keyboard navigation, visible focus, semantic tables/forms, useful errors, colour contrast and reduced-motion support.

53. Analytics

Keep product analytics privacy-conscious. Track product events such as:

```
account_registered
api_key_created
first_api_request_succeeded
endpoint_family_first_used
quota_80_percent
quota_exceeded
checkout_started
subscription_started
subscription_cancelled
```

Do not send API key secrets or raw customer request data to analytics vendors.

54. MVP build order for engineer

Milestone A — platform skeleton

- create Laravel 13 project;
- auth/dashboard shell;
- MySQL + Redis;
- API error/response infrastructure;
- request ID middleware;
- API key lifecycle;
- rate limiting;
- usage counter;
- OpenAPI skeleton.

Milestone B — geography utilities

- postcode value object;
- Postcodes.io adapter;
- postcode lookup/reverse lookup;
- council metadata;
- cache framework;
- provider health telemetry.

Milestone C — static/derived utilities

- bank holiday snapshot;
- working-day calculation;
- VAT utilities;
- SIC reference/search.

Milestone D — Companies House

- provider auth;
- company DTO/resource;
- profile/search/officers;
- error mapping;
- provider-wide limiter;

- tests/fixtures.

Milestone E — FSA / Police / Flood

Implement one provider at a time, including docs, telemetry and source register before moving on.

Milestone F — planning

- coordinate/postcode constraint query;
- curated constraint types;
- England coverage metadata;
- beta-provider fallback behaviour.

Milestone G — billing/dashboard

- plans;
- Cashier/Stripe;
- subscription webhook state;
- entitlements;
- usage screens;
- request logs;
- upgrade UX.

Milestone H — launch readiness

- load tests;
- security review;
- source terms review;
- production secrets;
- backups/restore rehearsal;
- monitoring/status;
- incident runbook;
- final OpenAPI validation.

55. Definition of done per endpoint

No endpoint should be considered complete until all of the following exist:

- input validation/value object where relevant;
- provider adapter method;
- DTO/normaliser;
- cache policy;
- source/error mapping;
- geographic scope metadata;
- source attribution entry;
- feature tests;
- provider fixture tests;
- OpenAPI schema;
- documentation page/examples;

- usage endpoint code;
- metrics tags;
- admin provider visibility;
- appropriate stale/failure policy.

This checklist prevents the API catalogue from becoming a collection of fragile one-off controllers.

56. Engineering risks

Risk: upstream schema changes

Mitigation: adapter mapping, fixture tests, live smoke probes and source-specific monitoring.

Risk: upstream rate limits

Mitigation: caching, provider-wide limiter, request coalescing and quotas.

Risk: source licensing changes

Mitigation: source register with periodic review and endpoint feature flags.

Risk: geographic inconsistency

Mitigation: explicit coverage metadata and no implied UK-wide claims.

Risk: cheap plan abused as bulk data export

Mitigation: per-key burst caps, monthly quotas, maximum page size and terms prohibiting abusive reconstruction where required by source terms.

Risk: Redis loss causes inaccurate quota

Mitigation: frequent durable rollup and reconciliation strategy.

Risk: a provider outage cascades to PHP worker saturation

Mitigation: aggressive timeouts, low retries, circuit breaker, stale cache and concurrency caps.

57. Decisions intentionally deferred

Do not decide prematurely:

- microservices;
- Kafka/event streaming;
- Elasticsearch/OpenSearch;
- Kubernetes;
- multi-region active/active;
- custom billing engine;
- paid enterprise SLA platform;
- warehouse/lake architecture;
- GraphQL;
- bespoke API gateway product.

Laravel + MySQL + Redis can comfortably support the MVP and significant early growth.

58. Launch checklist

Engineering:

- production config cache/routes optimized;
- APP_DEBUG=false;
- database migrations complete;
- Redis persistent enough for selected responsibilities;
- queue supervisor/Horizon healthy;
- scheduler running;
- health endpoints configured;
- secrets in managed secret store;
- TLS everywhere;
- backup/restore tested;
- admin 2FA enabled;
- log redaction verified;
- provider timeouts verified;
- provider quotas understood;
- feature flags tested;
- source attribution pages published;
- OpenAPI valid;
- smoke tests green.

Commercial/product:

- Stripe products/prices mapped to local plans;
- webhook endpoint live and signed;
- failed-payment/grace policy documented;
- quota emails tested;
- terms/privacy/acceptable-use published;
- independent/non-government affiliation statement visible;
- support contact available.

59. Recommended first production endpoint set

For the best ratio of perceived breadth to engineering risk, launch with:

```
POSTCODES
GET /v1/postcodes/{postcode}
GET /v1/postcodes/{postcode}/validate
GET /v1/coordinates/{lat}/{lng}/postcode

COUNCILS
GET /v1/councils/postcode/{postcode}

COMPANIES
GET /v1/companies/{number}
GET /v1/companies/search
GET /v1/companies/{number}/officers

DATES / UTILITIES
GET /v1/bank-holidays
GET /v1/working-days
GET /v1/vat/calculate
GET /v1/vat/remove
GET /v1/sic/{code}
GET /v1/sic/search

FOOD
GET /v1/food-establishments/{id}
GET /v1/food-establishments/search

CRIME
GET /v1/crime/nearby
GET /v1/crime/summary

FLOOD
GET /v1/flood/warnings
GET /v1/flood/nearby

PLANNING
GET /v1/planning/constraints
```

That is already enough for UKAPI.io to feel like a genuine toolbox rather than a single API with branding around it.

60. Final engineering recommendation

Implement UKAPI.io as a **modular Laravel 13 monolith** with a hard boundary between the public API contract and every upstream data source. Spend engineering effort on normalization, caching, metering, documentation and failure behaviour before spending it on infrastructure sophistication.

The platform's long-term technical asset is the normalized contract layer: provider quirks, licensing knowledge, geographic caveats, edge-case handling, source health behaviour and developer-friendly schemas. Protect that layer with interfaces, tests and observability from day one.

References and source notes

- Laravel 13 release notes: https://laravel.com/framework/docs/13.x/releases
- Laravel 13 deployment documentation: https://laravel.com/framework/docs/deployment
- Companies House API: https://developer.company-information.service.gov.uk/get-started
- Companies House Developer Hub: https://developer.company-information.service.gov.uk/
- GOV.UK reuse / Bank Holidays API: https://www.gov.uk/help/reuse-govuk-content
- Police.uk API: https://data.police.uk/docs/
- Police.uk street-level crime: https://data.police.uk/docs/method/crime-street/
- Food Standards Agency FHRS API: https://api.ratings.food.gov.uk/help
- Planning Data API: https://www.planning.data.gov.uk/docs
- Environment Agency flood-monitoring API: https://environment.data.gov.uk/flood-monitoring/doc/reference
- Postcodes.io: https://postcodes.io/docs/overview/

Source capabilities and terms should be rechecked immediately before production launch. This report reflects public documentation reviewed on 23 September 2026.